difficult to refactor the services. Over time, there might be a requirement to merge two services or split a service into two or more parts. If the client communicates directly, performing this kind of refactoring will be extremely difficult.

When an API gateway comes into the picture

A much better approach would be to use what is known as an API gateway. This is a single entry point to the system. It encapsulates the internal architecture and provides the service tailored to each client. It might also have other responsibilities that include authentication, monitoring, logging, caching, static response handling, etc.

An API gateway is responsible for request routing, composition and protocol translation. All the requests from the client go through an API gateway. It then routes requests to the appropriate microservice. The API gateway will often handle a request by invoking multiple microservices and then aggregate the results. It can translate between Web-friendly and Web-unfriendly protocols.

Here, in our example, the API gateway can provide an endpoint like */products/productId=xxx* that enables a client to retrieve all the product information (as well as other details) in a single request. In our case, the API gateway handles this request by invoking the various services like shipping, the recommendation service, reviews, etc.

Benefits and drawbacks

As you might expect, using an API gateway comes with both benefits and disadvantages. A major benefit is that it encapsulates the whole internal structure. Since rather than calling various services individually, clients call the API gateway, this reduces the number of round trips between the client and the application, and hence simplifies the client code.

On the down side, this is yet another component that must be developed, deployed and managed. Developers must update the gateway in order to expose each microservice endpoint. The process of updating should be lightweight. Otherwise, developers will be forced to wait in queue to update their service, which in turn may affect the business.

Despite these drawbacks, for most (almost all) real-world applications, it makes sense to use an API gateway.

Implementation

Let's look at the various design issues to consider when implementing an API gateway.

Performance and scalability: Only a handful of companies need to handle billions of requests per day. In any case, the performance and scalability of an API gateway is more important as this is the entry point. It is therefore necessary to build the API gateway on a platform that supports asynchronous and non-blocking I/O.

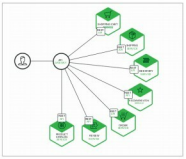

Figure 4: API gateway

Service invocation: As this is a distributed system, it must use an inter-process communication mechanism. There are two styles of IPC. One option is to use an asynchronous, message-based mechanism like JMS or AMQP. Another option is to use any synchronous mechanism such as HTTP.

Service discovery: The API gateway has to know the location of all microservices it needs to communicate with. Infrastructure services such as message brokers will have a static location. An API gateway can use two ways to determine the location—server side discovery and client side discovery. It is beyond the scope of this article to get into the details of these two types of discovery; yet, if the system uses client side discovery, the API gateway must be able to query the service registry to determine any service location.

Handling failures: This issue can arise in almost all distributed systems when one service calls another service that is responding slowly or is unavailable. For example, if the recommendation service is unresponsive in our *Product details* scenario, the API gateway should be able to serve the rest of the product details as they are still useful to the client. An unresponsive service can be returned by any default response.

The API gateway could also return cached data if it is available. An example is the product prices, for which changes are not frequent. Prices can be cached in the API gateway itself, in Redis or Memcached. By returning either default data or cached data, the gateway ensures that system failure does not affect the user experience.

For most microservices based applications, the API gateway, which acts as a single entry point, is an ideal solution. The gateway can also mask failures in the backend services, improve service messaging, as well as abstract some of the complexities and details of the microservice architecture.

Kong is a popular open source API gateway. NGINX Plus, Red Hat and 3scale each offer a free trial. END

By: A. Araskumar

The author works at HCL Technologies as a technical lead. He can be reached at *aaraskumar@yahoo.com*.